CliniFact

Clinical Research Fact-Verification
Project Summary & Research Report

Dataset | Methodology | Models | Limitations | Recommendations

1,970
total examples

3
evidence classes

1,260 / 316 / 394
train / val / test

Grounded only in the project's available code, datasets, configuration files and experiment outputs. Final model metrics are not yet available.

October 2026

Contents

1. Introduction . . . 3
2. Research Problem . . . 3
3. Main Project Features . . . 3
4. Data Sources . . . 4
5. Reference-Label Extraction . . . 4
6. Clinical-Trial Information Extraction . . . 4
7. Claim Generation . . . 5
8. Statistical Stance Labeling . . . 5
9. Combining Relevance and Stance . . . 5
10. Evidence Categories . . . 6
11. Dataset Construction . . . 6
12. PubMed Abstract Integration . . . 6
13. Dataset Size and Splits . . . 7
14. Class Distribution . . . 7
15. Machine-Learning Approach . . . 7
16. BioBERT Input Representation . . . 8
17. BioBERT Training Configuration . . . 8
18. Evaluation Metrics . . . 8
19. Available Experimental Results . . . 8
20. Large Language Model Component . . . 9
21. LLM Inference . . . 9
22. Potential Research Contributions . . . 9
23. Key Strengths . . . 10
24. Current Limitations . . . 10
25. Recommended Improvements . . . 10
26. Expected Research Outcomes . . . 12
27. Conclusion . . . 12

> **Scope.** This report is a concise, project-grounded summary based only on the available code, datasets, configuration files, and experiment outputs. The project does not contain one million words of documented research content.

1. Introduction

CliniFact is a clinical research fact-verification project. Its main purpose is to create a dataset that connects:

- Clinical trial protocols.
- Trial outcome measures.
- Intervention and comparator groups.
- Statistical results.
- Published research articles from PubMed.

The project focuses on determining whether a scientific claim is supported by published clinical evidence. It is designed to support:

- Clinical fact checking.
- Scientific claim verification.
- Evaluation of language models.
- Training biomedical language models.
- Analysis of the relationship between clinical-trial outcomes and publications.

The official project description is available in the `README.md`.

2. Research Problem

Clinical studies often contain:

- Complex intervention descriptions.
- Multiple treatment arms.
- Different outcome measures.
- Statistical significance values.
- References to several publications.

General-purpose language models may incorrectly:

- Interpret clinical outcomes.
- Associate the wrong publication with a trial.
- Treat background information as evidence.
- Confuse statistical significance with clinical superiority.

CliniFact addresses this problem by constructing structured claim–publication examples. Each example combines:

- A clinical claim generated from trial information.
- A PubMed article title and abstract.
- A factual evidence label.

3. Main Project Features

- Extraction of clinical-trial records from ClinicalTrials.gov JSON files.
- Identification of clinical-trial references and their types.

- Extraction of:
  - Trial identifiers.
  - Primary outcomes.
  - Outcome descriptions.
  - Time frames.
  - Intervention groups.
  - Comparator groups.
  - Statistical methods.
  - p-values.
  - Non-inferiority or equivalence types.
- Use of BioBERT embeddings to match clinical outcome groups with trial arm groups.
- Generation of structured clinical claims.
- Retrieval of PubMed article titles and abstracts.
- Creation of evidence labels.
- Construction of training, validation, and test datasets.
- Training and evaluation scripts for:
  - BERT, RoBERTa, BioBERT, PubMedBERT, and other transformer-based models.
  - Large language models using LoRA and quantization.

4. Data Sources

The project uses two primary sources:

Source	Provides
ClinicalTrials.gov	Trial protocol information; intervention and comparator details; outcome measures; statistical analysis information; reference classifications.
PubMed	Article titles; article abstracts; publication identifiers (PMIDs).

The project retrieves PubMed information using the `metapub` package in `g_create_dataset_with_all_inputs_part2.py`.

5. Reference-Label Extraction

The script `a_extract_relevancy_label.py` extracts references from clinical-trial JSON files and classifies them into three categories:

Category	Meaning	Numeric label
`RESULT`	Publication reporting trial results	1
`BACKGROUND`	Publication providing background information	0
`DERIVED`	Publication derived from or related to the trial	2

Only references with valid PubMed identifiers are retained.

6. Clinical-Trial Information Extraction

The script `b_extract_clinical_trials.py` extracts information from clinical-trial records, focusing on trials with two arm groups. It collects:

- NCT identifier.

- Outcome type, title, description, and time frame.
- p-values and statistical methods.
- Non-inferiority types.
- Intervention-group and comparator-group information.

BioBERT is used to calculate semantic similarity between outcome-measure groups and clinical-trial arm-group labels. The most similar arm group is selected for each outcome group, which allows the system to identify which clinical outcome group corresponds to which intervention or comparator.

7. Claim Generation

The project creates claims from the trial intervention, comparator, and outcome information. A typical claim follows the structure:

> **Claim template. Intervention group** is **superior, equivalent, or non-inferior** to the **comparator group** for a specific **clinical outcome** during a given **time frame**.

Claim components include:

- Intervention title.
- Comparator title.
- Outcome title.
- Outcome time frame.
- Non-inferiority or superiority type.

The dataset therefore transforms structured clinical-trial information into natural-language scientific claims.

8. Statistical Stance Labeling

The script `c_extract_stance_label.py` determines the stance of an outcome claim. The current rule is based mainly on p-values:

Condition	Stance	Interpretation
p-value ≤ `0.05`	Positive	Supporting evidence
p-value > `0.05`	Negative	Inconclusive evidence

The method examines multiple p-values and assigns a positive label if at least one qualifying p-value is found. This creates an automatic statistical labeling method without requiring manual annotation for every example.

9. Combining Relevance and Stance

The script `d_intersection_relevancy_stance_label.py` combines clinical-trial reference labels with statistical stance labels. For each trial, it collects:

- PMIDs associated with result publications.
- PMIDs associated with background publications.

The combined records are used to determine whether a publication supports, contradicts, or does not provide information about a claim.

10. Evidence Categories

The script `e_evidence_labels4primary_outcome.py` generates the primary evidence labels. The project uses three main classes:

Class	Assigned when	Meaning
Evidence	Primary outcome; exactly one result publication is associated with the trial; stance label is positive.	The claim is supported by evidence.
Inconclusive	Primary outcome; exactly one result publication is associated with the trial; stance label is negative.	Extracted statistical evidence does not support a positive conclusion.
No Information	Primary outcome; one or more background publications are available.	The publication is background information, not direct evidence for the claim.

11. Dataset Construction

The script `f_create_dataset_with_all_inputs.py` creates structured dataset records. Each record can include:

- NCT identifier and PMID.
- Outcome information.
- Intervention title and description.
- Comparator title and description.
- Arm-group types.
- p-values, statistical methods, and non-inferiority types.
- Evidence label.

Duplicate records are removed by comparing complete JSON objects, producing a more consistent dataset for machine-learning experiments.

12. PubMed Abstract Integration

The project retrieves article titles and abstracts using PubMed identifiers. The publication text is joined with the clinical claim, so the resulting example contains:

- A generated clinical claim.
- A scientific article title.
- A scientific article abstract.
- A three-class evidence label.

This allows models to compare the claim against the content of the associated publication.

13. Dataset Size and Splits

The repository contains the following processed dataset sizes (split implemented in `h_train_valid_test_split.py`):

Split	Examples	Share
Training	1,260	≈ 64%
Validation	316	≈ 16%
Test	394	≈ 20%
Total	**1,970**	100%

Table 1. Processed dataset splits.

14. Class Distribution

Label	Class	Training	Validation	Test
0	Inconclusive	262	67	54
1	Evidence	366	83	113
2	No information	632	166	227
	Total	**1,260**	**316**	**394**

Table 2. Label distribution per split.

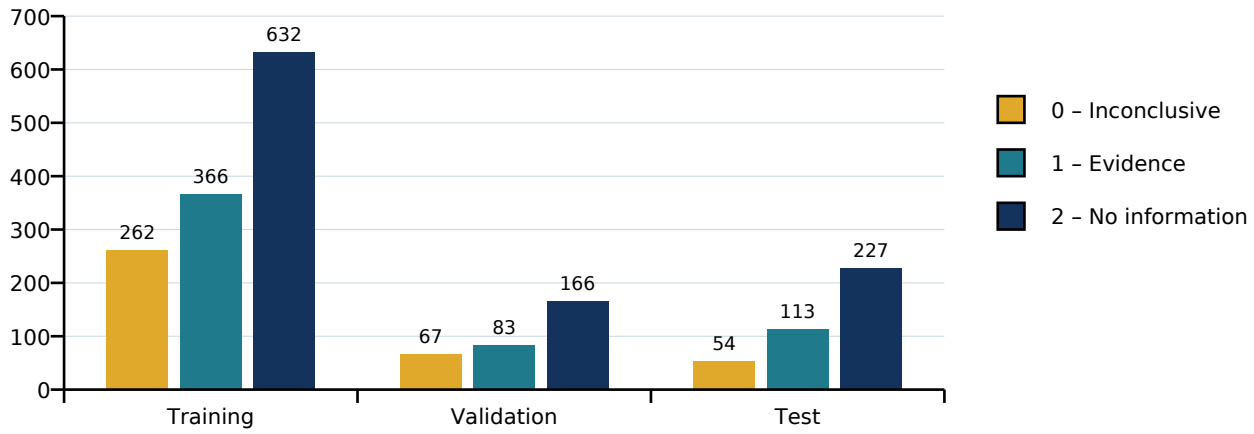


Figure 1. Class distribution across training, validation and test sets.

Important observation

- The dataset is imbalanced.
- The `no information` class is the largest class in all three splits.
- The `inconclusive` class is relatively smaller.
- Accuracy alone may therefore be misleading.
- Macro-F1 is an important evaluation metric because it gives equal importance to all classes.

15. Machine-Learning Approach

- The project includes transformer-based classification experiments.

- Candidate models include BERT, RoBERTa, BioBERT, PubMedBERT, and sentence-transformer models.
- The main baseline experiment uses `dmis-lab/biobert-base-cased-v1.1`.
- BioBERT is suitable because it is pretrained on biomedical and scientific language.
- The classification task contains three output classes: Evidence, Inconclusive, and No information.

16. BioBERT Input Representation

The classification input combines the generated claim, the article title, and the article abstract.

- The claim is formatted using clinical information: intervention, comparator, outcome, time frame, and superiority or equivalence type.
- The article title and abstract form the evidence text.
- The input is tokenized using a maximum sequence length of 512 tokens.
- The model can truncate long abstracts.
- The training implementation is in `biobert_baseline.py`.

17. BioBERT Training Configuration

The available BioBERT configuration (documented in `config.json`) specifies:

Parameter	Value
Model	`dmis-lab/biobert-base-cased-v1.1`
Learning rate	`2e-5`
Training batch size	32
Evaluation batch size	128
Micro-batch size	8
Gradient accumulation steps	4
Epochs (baseline configuration)	3
Early-stopping patience	10
CPU threads	4
Checkpoint selection	Highest validation macro-F1
Dataset sizes	Train 1,260 | Validation 316 | Test 394

18. Evaluation Metrics

- The project uses accuracy, macro-F1, precision, recall, and classification reports.
- The baseline experiment selects the best checkpoint using validation macro-F1.
- Test evaluation is intended to report test accuracy, test macro-F1, and per-class precision, recall and F1-score.
- The evaluation code is available in `i_eval.py`.

19. Available Experimental Results

The repository contains:

- Dataset distribution tables.
- Label distribution figures.
- Outcome-type, intervention-type and publication-year distributions.
- Text-length visualizations.
- BioBERT configuration files.

> **Important.** The repository does **not** contain populated numerical test metrics. The available `metrics_summary.json` holds an empty object (`{}`), so there are no verified final accuracy or macro-F1 values. A research paper should not claim a specific final model performance unless training and evaluation are rerun and the generated metric files are saved.

The available visualization outputs are located in the BioBERT visualization directory.

20. Large Language Model Component

The script `llm_finetune.py` supports:

- Causal language models.
- LoRA parameter-efficient fine-tuning.
- 4-bit and 8-bit quantization.
- Gradient accumulation.
- Training and validation datasets.
- Hugging Face model loading.

The default configuration refers to a large Llama-based model. The LLM is trained to produce one of three labels (`TRUE`, `FALSE`, `NONE`), which the project maps to the classification categories.

21. LLM Inference

The script `llm_inference.py` performs inference on test prompts. It obtains probabilities for `TRUE`, `FALSE` and `NONE`, selects the most probable label, and converts it to a numeric label:

Generated label	Classification	Meaning
`TRUE`	Evidence	Claim supported
`FALSE`	Inconclusive	Claim not supported by statistics
`NONE`	No information	Background only

The predictions are evaluated using a classification report.

22. Potential Research Contributions

- A structured dataset for clinical evidence verification.
- Integration of clinical-trial records and PubMed literature.
- Automatic generation of clinical claims from trial outcomes.
- A three-class evidence classification task.
- Biomedical language-model evaluation.
- A framework for comparing statistical evidence, publication relevance, and claim-level factual consistency.
- A possible foundation for clinical decision-support and scientific literature-analysis tools.

23. Key Strengths

- Uses real clinical-trial and biomedical publication data.
- Combines structured and unstructured information.
- Uses domain-specific BioBERT representations.
- Includes intervention, comparator, outcome, and statistical details.
- Separates direct evidence from background information.
- Provides reproducible train, validation, and test splits.
- Includes both encoder-based and generative-model approaches.
- Includes visualization scripts for dataset analysis.
- Uses macro-F1 for checkpoint selection, which is appropriate for imbalanced classes.

24. Current Limitations

- The dataset is relatively small for training large language models.
- The class distribution is imbalanced.
- Labels are generated using heuristic rules rather than extensive expert annotation.
- A p-value threshold of `0.05` does not fully represent:
  - Clinical importance.
  - Effect size.
  - Confidence intervals.
  - Study quality.
  - Bias.
  - Sample size.
- A statistically significant result does not always mean that a clinical claim is practically meaningful.
- Background publications are treated as `no information`, although some may contain relevant supporting information.
- The data pipeline depends on external PubMed retrieval.
- Missing abstracts or invalid PMIDs may result in incomplete examples.
- The available experiment directory does not contain final populated performance metrics.
- Several scripts contain hard-coded absolute paths, reducing portability.
- The LLM inference script requires configuration cleanup before public or reproducible use.
- The matching approach selects the most similar arm group but has no confidence threshold or human verification step.
- Inter-annotator agreement is not documented because labels are primarily programmatically generated.

25. Recommended Improvements

Dataset improvements

- Increase the number of clinical trials and publications.
- Add more medical specialties and disease areas.
- Include multiple publications per clinical trial.
- Preserve publication-level relationships instead of restricting some cases to one PMID.
- Add metadata such as sample size, effect size, confidence interval, study design, risk of bias, and follow-up duration.

- Add manually reviewed gold-standard labels.
- Record uncertainty for automatically generated labels.

Labeling improvements

- Replace the simple p-value rule with richer statistical reasoning.
- Include effect direction and magnitude.
- Distinguish supporting, refuting, mixed, insufficient, and irrelevant evidence.
- Use expert clinical reviewers to validate a representative sample.
- Measure agreement using Cohen's kappa or Krippendorff's alpha.
- Apply label-quality checks to identify ambiguous examples.

Modeling improvements

- Compare BioBERT with PubMedBERT and modern biomedical encoder models.
- Use class-weighted loss or focal loss to address class imbalance.
- Report macro-F1, weighted-F1, balanced accuracy, and per-class metrics.
- Use stratified splitting, and consider grouping splits by trial identifier to prevent information leakage.
- Add cross-validation for more reliable performance estimates.
- Evaluate claim-only, abstract-only, and claim-plus-abstract inputs separately.
- Use evidence retrieval before classification.
- Add contradiction detection and natural-language inference models.
- Experiment with long-context biomedical models for full articles.

Retrieval improvements

- Retrieve full-text articles when legally and technically available.
- Rank evidence sentences instead of using only complete abstracts.
- Use dense retrieval and biomedical vector search.
- Add citation-level and sentence-level evidence alignment.
- Return supporting sentences with each prediction.
- Provide confidence scores and an explanation for every classification.

Reproducibility improvements

- Replace hard-coded paths with command-line arguments or configuration files.
- Add a requirements file or environment specification.
- Document the exact raw-data acquisition process.
- Store random seeds and software versions.
- Save model checkpoints and final metric files.
- Make the pipeline executable from the repository root.
- Add unit tests for p-value parsing, label assignment, duplicate removal, dataset splitting, and claim construction.
- Remove placeholder authentication values and use secure environment variables.

Evaluation improvements

- Publish final test-set metrics.
- Include confusion matrices.
- Report confidence intervals for performance.

- Compare against majority-class and random baselines, TF-IDF plus classical classifiers, BERT-based baselines, BioBERT, PubMedBERT, and fine-tuned LLMs.
- Perform error analysis by outcome type, intervention type, publication year, abstract length, evidence class, and statistical method.
- Conduct an ablation study to measure the contribution of claim text, article title, article abstract, p-values, and intervention/comparator descriptions.

26. Expected Research Outcomes

- The project is expected to show whether biomedical language models can classify the relationship between clinical claims and research publications.
- A successful model should:
  - Identify direct supporting evidence.
  - Recognize inconclusive results.
  - Separate background information from direct evidence.
  - Generalize across different clinical domains.
- The research can contribute to safer clinical literature analysis by reducing unsupported interpretations of trial results.
- The strongest conclusion must be based on completed test metrics and expert validation, which are not currently available in the repository.

27. Conclusion

CliniFact presents a pipeline for creating and evaluating clinical evidence-verification datasets. It combines:

- ClinicalTrials.gov trial data.
- PubMed publications.
- Statistical outcome analysis.
- BioBERT semantic matching.
- Transformer classification.
- LLM fine-tuning.

The project currently provides a 1,970-example processed dataset with three evidence categories. Its main research value is the connection between structured clinical-trial claims and unstructured biomedical literature.

The primary next step is to complete reproducible model training, save verified evaluation metrics, validate labels with clinical experts, and improve the statistical and evidence-retrieval methodology.